INFOESTE • OFICINA PRÁTICA

Introdução à Cibersegurança na Prática

Conceitos fundamentais, proteção de dados e atividades práticas



- Senhas e autenticação
- Confidencialidade e integridade
- Hash e criptografia
- Engenharia social e phishing
- Atividades práticas em Python

Gabriel Savoldi
Leandro Paes
Maria Eduarda Ribeiro
Natalia Viudes

INFOESTE

O que vamos aprender?

Da teoria à prática

Fundamentos de segurança:

Entenderemos quais características de uma informação precisam ser protegidas e quais tipos de ataques podem comprometer um sistema.

Identidade e acesso:

Veremos como sistemas identificam usuários, por que senhas são utilizadas e como permissões controlam o que cada pessoa pode fazer.

Proteção das informações:

Conheceremos conceitos de hash e criptografia e veremos como eles podem ser utilizados para proteger dados.

O fator humano:

Analisaremos golpes de phishing e situações em que o atacante tenta enganar uma pessoa em vez de atacar diretamente um sistema.

Prática:

Durante o curso, desenvolveremos pequenos programas e resolveremos desafios de segurança.

Segurança da Informação

O que exatamente estamos tentando proteger?

Segurança da informação é o conjunto de medidas utilizadas para proteger informações, sistemas e recursos contra ações que possam comprometer sua segurança.

A informação pode estar armazenada em um computador, celular, servidor ou banco de dados, mas também pode estar sendo transmitida por uma rede.

Um problema de segurança pode ocorrer quando alguém acessa, modifica, destrói ou impede o acesso a uma informação sem autorização.

Exemplos:

- Roubo de uma senha
- Vazamento de informações
- Alteração de um arquivo
- Indisponibilidade de um sistema

O que é um ataque à segurança?

Nem todo ataque acontece da mesma forma

Tópico 1:

Um ataque ocorre quando uma ação compromete algum aspecto da segurança de uma informação, sistema ou comunicação.

Tópico 2:

O objetivo de um atacante pode ser apenas obter informações, mas também pode envolver modificar dados, interromper sistemas ou se passar por outra pessoa.

Tópico 3:

Os ataques podem ser divididos em dois grandes grupos:

- Ataques passivos
- Ataques ativos

Ataques Passivos

O atacante observa, mas não interfere

Tópico 1:

Um ataque passivo busca obter informações sem alterar o funcionamento do sistema ou o conteúdo da comunicação.

Tópico 2:

O atacante pode tentar observar mensagens, arquivos ou padrões de comunicação.

Tópico 3:

Como o sistema continua funcionando normalmente, esse tipo de ataque pode ser difícil de perceber.

Exemplos:

- Leitura não autorizada de uma mensagem
- Observação do tráfego de uma rede
- Monitoramento de uma comunicação

Ataques Ativos

O atacante interfere no sistema

Tópico 1:

Nos ataques ativos, existe uma modificação ou perturbação do sistema, de seus recursos ou de uma comunicação.

Tópico 2:

O atacante pode alterar informações, inserir novos dados, interromper um serviço ou tentar assumir a identidade de outra pessoa.

Exemplos:

- Alteração de mensagens
- Modificação de arquivos
- Negação de serviço
- Repetição de mensagens
- Disfarce de identidade

Serviços de Segurança

Que propriedades queremos garantir?

Tópico 1:

Um serviço de segurança é uma funcionalidade utilizada para aumentar a proteção dos recursos de um sistema e das informações de uma organização.

Tópico 2:

Cada serviço procura resolver um tipo diferente de problema de segurança.

Principais serviços estudados:

- Confidencialidade
- Autenticação
- Integridade
- Irretratabilidade
- Disponibilidade
- Controle de acesso

Serviço de Confidencialidade

Protegendo informações contra divulgação não autorizada

Tópico 1:

A confidencialidade busca impedir que uma informação seja conhecida por alguém que não possui autorização.

Tópico 2:

A proteção deve existir tanto quando a informação está armazenada quanto quando está sendo transmitida.

Tópico 3:

A entidade não autorizada pode ser uma pessoa, um programa, outro sistema ou até outro equipamento.

Exemplo:

Uma conversa privada não deveria poder ser lida por uma pessoa que não participa da conversa.

Serviço de Integridade

A informação continua sendo a mesma?

Tópico 1:

Integridade significa conseguir determinar se uma informação foi modificada por uma entidade não autorizada.

Tópico 2:

Uma informação íntegra deve continuar correspondente aquela que foi originalmente gerada por uma entidade autorizada.

Tópico 3:

Podem existir problemas de integridade quando ocorre:

- Modificação
- Inserção
- Exclusão
- Repetição de informações

Exemplo:

Mensagem original:
Transferir R$ 100

Mensagem alterada:
Transferir R$ 900

Mesmo que ninguém além das pessoas autorizadas tenha visto a mensagem, sua integridade foi comprometida.

Serviço de Disponibilidade

Um sistema seguro também precisa funcionar

Tópico 1:

Disponibilidade significa garantir que um recurso esteja acessível e utilizável quando usuários autorizados precisarem dele.

Tópico 2:

Não adianta um sistema proteger perfeitamente seus dados se ninguém conseguir utilizá-lo.

Tópico 3:

Ataques de negação de serviço procuram justamente prejudicar ou impedir o funcionamento de determinado serviço.

Exemplo:

O sistema da escola possui todos os dados corretamente protegidos, mas fica fora do ar durante o período de matrícula.

Confidencialidade, Integridade e Disponibilidade

Três perguntas importantes sobre uma informação

Confidencialidade:

Somente entidades autorizadas devem conseguir visualizar determinada informação.

Integridade:

Precisamos conseguir determinar se uma informação foi modificada indevidamente.

Disponibilidade:

Informações e sistemas devem estar disponíveis e utilizáveis quando entidades autorizadas precisarem deles.

Resumo:

Confidencialidade → Quem pode VER?
Integridade → A informação foi ALTERADA?
Disponibilidade → Consigo ACESSAR quando preciso?

Qual propriedade foi comprometida?

Confidencialidade, Integridade ou Disponibilidade?

SITUAÇÃO 1

Uma pessoa conseguiu ler mensagens privadas de outro usuário.

SITUAÇÃO 2

Um atacante alterou a nota de um aluno no sistema.

SITUAÇÃO 3

O servidor da escola ficou inacessível durante todo o dia.

SITUAÇÃO 4

Um arquivo foi alterado sem autorização, mas ninguém percebeu inicialmente.

Qual propriedade foi comprometida?

Confidencialidade, Integridade ou Disponibilidade?

SITUAÇÃO 1

Uma pessoa conseguiu ler mensagens privadas de outro usuário.

SITUAÇÃO 2

Um atacante alterou a nota de um aluno no sistema.

SITUAÇÃO 3

O servidor da escola ficou inacessível durante todo o dia.

SITUAÇÃO 4

Um arquivo foi alterado sem autorização, mas ninguém percebeu inicialmente.

1 → Confidencialidade **2 → Integridade**
3 → Disponibilidade **4 → Integridade**

Serviço de Autenticação

Como o sistema sabe quem você é?

Tópico 1:

Autenticação é o processo utilizado para verificar se uma entidade realmente é quem afirma ser.

Tópico 2:

Em um sistema, a autenticação de usuário procura comprovar que aquela pessoa está autorizada a utilizar determinada identidade.

Tópico 3:

Uma forma comum de autenticação utiliza:

Usuário + Senha

Pergunta principal da autenticação:

“Quem é você e como consegue provar isso?”

Autenticação ≠ Controle de Acesso

Entrar no sistema não significa poder fazer tudo

Autenticação:

Determina quem é o usuário.

Pergunta:
Quem é você?

Controle de acesso:

Determina quais recursos o usuário pode acessar e quais operações possui permissão para realizar.

Pergunta:
O que você pode fazer?

Exemplo:

ALUNO
✅ consultar nota
❌ alterar nota

PROFESSOR
✅ consultar nota
✅ alterar nota

Senhas como forma de autenticação

Uma senha funciona como uma prova de conhecimento

Tópico 1:

Uma senha é uma informação secreta utilizada para demonstrar que determinado usuário conhece uma credencial associada à conta.

Tópico 2:

Se outra pessoa descobrir a senha, o sistema pode ter dificuldade para diferenciar o usuário legítimo do atacante.

Tópico 3:

Por isso, uma senha deve ser difícil de prever e não deve ser compartilhada.

Exemplos de senhas previsíveis:

```
123456
senha123
admin123
madu2008
```

Criando senhas mais resistentes

Quanto menos previsível, melhor

Tópico 1:

Senhas muito curtas possuem menos possibilidades e podem ser testadas com maior facilidade.

Tópico 2:

Informações pessoais, como nome, aniversário, time ou nome de animais, podem ser previsíveis.

Tópico 3:

Reutilizar a mesma senha em vários serviços aumenta o impacto de um vazamento.

Boas práticas:

- Utilizar senhas longas
- Evitar informações óbvias
- Não reutilizar a mesma senha
- Utilizar autenticação adicional quando disponível

Observação:

Os critérios que utilizaremos no programa a seguir são didáticos e não representam sozinhos uma avaliação completa da segurança de uma senha.

Ataques contra credenciais

Nem sempre existe uma “invasão” sofisticada

Adivinhação:

O atacante tenta informações previsíveis relacionadas ao usuário.

Tentativas automatizadas:

Programas podem testar grandes quantidades de possibilidades.

Reutilização de credenciais:

Uma senha obtida em um vazamento pode ser testada em outros serviços.

Engenharia social:

Em vez de descobrir a senha tecnicamente, o atacante tenta convencer a própria vítima a fornecê-la.

Projeto: Analisador de Senhas

Vamos aplicar algumas regras utilizando Python

Objetivo: Criar um programa que analise determinadas características de uma senha.

O programa verificará:

- Quantidade de caracteres
- Existência de letras maiúsculas
- Existência de letras minúsculas
- Existência de números
- Existência de caracteres especiais

Exemplo esperado:

```
Digite sua senha: Exemplo123!

Tamanho adequado: ✅
Maiúscula: ✅
Minúscula: ✅
Número: ✅
Caractere especial: ✅
```

Construindo o analisador

Verificando cada característica da senha

exemplo.py

```
senha = input("Digite uma senha: ")

pontos = 0

if len(senha) >= 12:
    pontos += 1

if any(c.isupper() for c in senha):
    pontos += 1

if any(c.islower() for c in senha):
    pontos += 1

if any(c.isdigit() for c in senha):
    pontos += 1

if any(not c.isalnum() for c in senha):
    pontos += 1

print("Pontuação:", pontos)
```

Abaixo do código:

len() → quantidade de caracteres
isupper() → verifica letra maiúscula
islower() → verifica letra minúscula
isdigit() → verifica número
isalnum() → verifica letra ou número

Agora é com vocês

Classifique a senha

Enunciado: Utilizando a pontuação calculada pelo programa, adicione uma classificação.

Regras:

0 a 2 pontos → Senha FRACA

3 ou 4 pontos → Senha MÉDIA

5 pontos → Senha FORTE

Bônus:

Exiba uma mensagem específica caso a senha tenha menos de 8 caracteres.

Tempo sugerido: 10 a 15 minutos.

Como um sistema deve guardar senhas?

Salvar a senha diretamente é perigoso

Imagine este banco de dados:

USUÁRIO	SENHA
joao	joao123
maria	banana123
pedro	abc123

Tópico 1:

Se uma pessoa obtiver acesso ao banco de dados, todas as senhas ficarão imediatamente visíveis.

Tópico 2:

Por isso, sistemas não deveriam depender do armazenamento da senha original em texto puro.

Pergunta: Como verificar uma senha sem precisar guardar diretamente seu conteúdo original?

O que é um Hash?

Uma representação calculada a partir dos dados

Tópico 1:

Uma função hash recebe uma informação de entrada e produz um valor de tamanho determinado.

Tópico 2:

A mesma entrada gera o mesmo resultado quando utilizamos o mesmo algoritmo.

Tópico 3:

Uma pequena alteração na entrada produz um resultado diferente.

Tópico 4:

Uma função hash não é criada para que seu resultado seja simplesmente revertido para recuperar a entrada original.

Exemplo:

"banana123"
↓
SHA-256
↓
valor hash

Hash × Criptografia

Dois conceitos diferentes

Hash	Criptografia
Não é projetado para ser revertido	Existe um processo de decifração
Muito utilizado para verificações	Utilizada para esconder informações
Pode auxiliar na integridade	Busca confidencialidade
Exemplo: SHA-256	Exemplo: AES

Resumo:

HASH
Entrada → Resultado

CRIPTOGRAFIA
Mensagem → Texto cifrado → Mensagem

Calculando SHA-256

Utilizando a biblioteca hashlib

exemplo.py

```
texto1 = input("Digite um texto (ou Enter para 'ola'): ") or "ola"

hash1 = hashlib.sha256(texto1.encode("utf-8")).hexdigest()

print("Hash:", hash1)
```

Explicações:

hashlib
Biblioteca do Python com funções de hash.

encode()
Transforma o texto em bytes.

sha256()
Calcula o hash utilizando SHA-256.

hexdigest()
Representa o resultado em formato hexadecimal.

Hash também pode verificar integridade

O arquivo continua sendo o mesmo?

Arquivo original / Arquivo alterado:

Arquivo original:
Transferir R$ 100

↓ SHA-256
Hash A

Arquivo alterado:
Transferir R$ 900

↓ SHA-256
Hash B

Conclusão:

Se o conteúdo do arquivo mudar, o valor calculado também muda.

Isso permite comparar o estado atual de um arquivo com um valor conhecido anteriormente.

Verificador de Integridade

Calculando o hash de um arquivo

integridade.py

```
texto1 = input("Digite um texto (ou Enter para 'ola'): ") or
"ola"
texto2 = input("Digite uma pequena variacao (ou Enter para
'Ola'): ") or "Ola"

hash1 = hashlib.sha256(texto1.encode("utf-8")).hexdigest()
hash2 = hashlib.sha256(texto2.encode("utf-8")).hexdigest()

print("Hash 1:", hash1)
print("Hash 2:", hash2)
print("Sao iguais?", hmac.compare_digest(hash1, hash2))
```

Atividade:

hmac
Biblioteca do Python utilizada para autenticação e comparação segura de dados.

hmac.compare_digest()
Compara os dois hashes e retorna True se forem iguais ou False se forem diferentes.

Pergunta: O arquivo continua possuindo a mesma integridade em relação à versão anterior?

Hash comum não é suficiente para senhas

Senhas possuem necessidades específicas

Tópico 1:

Algoritmos como SHA-256 foram desenvolvidos para calcular hashes rapidamente.

Tópico 2:

Para armazenamento de senhas, normalmente queremos dificultar a execução de enormes quantidades de tentativas.

Tópico 3:

Por isso existem mecanismos próprios para processamento de senhas.

Exemplos:

PBKDF2;
scrypt;
Argon2.

Fluxo simplificado:

Senha
+
Salt
↓
Função apropriada para senhas
↓
Valor armazenado

E se dois usuários possuírem a mesma senha?

Mesma entrada produz o mesmo hash

Exemplo:

João
Senha: banana123
Hash: ABCD...

Maria
Senha: banana123
Hash: ABCD...

Problema:

É possível perceber que os dois usuários possuem entradas iguais mesmo sem conhecer diretamente a senha.

Solução utilizada em sistemas de senhas:

Adicionar um valor aleatório diferente para cada usuário.

Esse valor é chamado de salt.

O que é Salt?

Adicionando aleatoriedade ao processamento da senha

Salt é um valor aleatório associado a uma senha antes do processo utilizado para gerar o valor armazenado.

Exemplo: João

João

senha: banana123
salt: A8F3

↓ processamento

Resultado: 82AC...

Exemplo: Maria

Maria

senha: banana123
salt: 91BD

↓ processamento

Resultado: 72F1...

Conclusão: Mesmo utilizando a mesma senha, os valores finais podem ser diferentes.

Criptografia e Confidencialidade

Como esconder uma informação?

Tópico 1:

Hash permite verificar informações, mas não é utilizado para recuperar posteriormente o conteúdo original.

Tópico 2:

Existem situações em que precisamos proteger uma informação e depois conseguir recuperá-la.

Tópico 3:

Para isso podemos utilizar criptografia.

Exemplos:

- Mensagens
- Arquivos
- Dados transmitidos por redes
- Informações armazenadas

Criptografia Simétrica

Transformando texto legível em texto cifrado

Os cinco elementos principais são:

1. Texto legível:

Mensagem ou informação original.

2. Algoritmo de cifração:

Realiza as transformações sobre os dados.

3. Chave secreta:

Valor utilizado pelo algoritmo durante o processo de cifração.

4. Texto cifrado:

Resultado embaralhado e não inteligível.

5. Algoritmo de decifração:

Utiliza o texto cifrado e a chave para recuperar a informação original.

Da mensagem ao texto cifrado

A chave participa do processo

Fluxo:

Mensagem original
+
Chave K
↓
Cifração
↓
Texto cifrado
↓
Decifração
+
Chave K
↓
Mensagem original

Tópico:

Na criptografia simétrica, emissor e receptor compartilham uma chave secreta.

O algoritmo pode ser conhecido

O segredo deve estar na chave

Tópico 1:

Um algoritmo criptográfico seguro não deve depender do fato de ninguém conhecer seu funcionamento.

Tópico 2:

Mesmo conhecendo o algoritmo, uma pessoa sem a chave adequada não deveria conseguir recuperar a informação de maneira viável.

Tópico 3:

Emissor e receptor precisam manter a chave secreta protegida.

Problema importante:

Como entregar essa chave de maneira segura para a outra pessoa?

Entendendo uma transformação com XOR

Um exemplo simples para visualizar a ideia

Tabela XOR:

A	B	Resultado
0	0	0
0	1	1
1	0	1
1	1	0

Exemplo:

```
Mensagem:  1 0 1 0
Chave:     1 1 0 0
           ———————
Resultado: 0 1 1 0
```

Aplicando novamente a mesma chave:

```
Resultado: 0 1 1 0
Chave:     1 1 0 0
           ———————
Mensagem:  1 0 1 0
```

Observação: Este é apenas um exemplo didático para compreender uma transformação reversível, não uma implementação segura de criptografia moderna.

Da mensagem ao texto cifrado

Prática

Fluxo:

```
texto = (input("Mensagem ficticia (Enter =
SEGREDO): ") or "SEGREDO").upper()
    cifrado = cifra_cesar(texto, 3)
    print("Texto original:", texto)
    print("Texto cifrado (+3):", cifrado)
    print("Texto decifrado (-3):",
cifra_cesar(cifrado, -3))
    try:
        from cryptography.fernet import Fernet
    except Exception:
        print("Criptografia moderna extra
indisponivel: biblioteca opcional nao
instalada.")
    else:
```

Fluxo:

```
    else:
        chave = Fernet.generate_key()
        sistema = Fernet(chave)
        token =
sistema.encrypt(texto.encode("utf-8"))
        print("\nEXTRA - Criptografia
autenticada com Fernet:")
        print("Texto cifrado (inicio):",
token.decode("ascii")[:50] + "...")
        print("Decifrado:",
sistema.decrypt(token).decode("utf-8"))
        print("Importante: proteger e guardar a
chave de forma segura.")
```

MISSÃO FINAL

Missão Final: Gerador de senhas

Agora vocês são a equipe de segurança

Cenário: Uma empresa deseja melhorar a segurança das contas de seus funcionários. Sua missão é desenvolver um aplicativo que gere senhas aleatórias e difíceis de adivinhar.

O aplicativo deverá permitir

- Escolher o tamanho da senha (4 a 32 caracteres).
- Incluir números e símbolos.
- Gerar senhas aleatórias automaticamente.
- Copiar a senha gerada.
- Exibir uma classificação didática da senha.

Objetivo:

Desenvolver um gerador de senhas com interface gráfica utilizando Python, aplicando os conceitos de segurança aprendidos durante a aula.

Conceitos que vamos aplicar:

- Aleatoriedade: gerar caracteres utilizando a biblioteca secrets.
- Personalização: permitir escolher o tamanho e os tipos de caracteres.
- Interface gráfica: utilizar tkinter para interagir com o usuário.
- Avaliação: classificar as senhas conforme critérios definidos no programa.

E se o alvo não for o computador?

Pessoas também fazem parte da segurança

Tópico 1:

Nem todo ataque depende de explorar uma falha de programação ou de um sistema.

Tópico 2:

Um atacante também pode tentar manipular uma pessoa para conseguir informações ou induzi-la a executar determinada ação.

Situações comuns:

- Criar sensação de urgência
- Fingir ser alguém conhecido
- Fazer ameaças
- Oferecer prêmios
- Pedir senhas ou códigos
- Induzir o usuário a acessar um link

Frase de destaque:

Um sistema pode estar tecnicamente protegido e ainda assim ser comprometido por decisões humanas.

Phishing

Tentando enganar a vítima

Exemplo de mensagem:

ATENÇÃO!

Sua conta será bloqueada nas próximas horas.

Confirme imediatamente seus dados:

conta-segura-verificacao.xyz

Possíveis sinais de alerta:

- Urgência exagerada
- Ameaças
- Pedido inesperado de informações
- Endereço estranho
- Link diferente do serviço verdadeiro
- Contexto inesperado

Pergunta para a turma: O que nessa mensagem faria vocês desconfiarem?

ENGENHARIA SOCIAL

Phishing ou Confiável?

Analise antes de clicar

Regras: As equipe receberam diferentes mensagens.

Para cada mensagem, deverá decidir:

DECISÃO

🟢 Parece confiável

ou

🔴 Parece suspeita

Pontuação:

Classificação correta → 1 ponto

Justificativa correta → 1 ponto

Objetivo:

Não basta decidir. A equipe precisa explicar quais elementos da mensagem influenciaram a decisão.

Phishing

Tentando enganar a vítima

Exemplo de mensagem:

Mensagem 1 — Instagram

"AVISO URGENTE!

Sua conta do Instagram será desativada em 30 minutos devido a uma atividade suspeita. Para evitar o bloqueio, confirme sua identidade em: instagram-seguranca.example"

Exemplo de mensagem:

Mensagem 2 — Escola

"Olá, alunos! O calendário das avaliações foi atualizado. As novas datas estão disponíveis na seção 'Avisos' do aplicativo oficial da escola."

Recebida dentro do aplicativo oficial da escola.

O que nessa mensagem faria vocês desconfiarem?

ENGENHARIA SOCIAL

Phishing

Tentando enganar a vítima

Exemplo de mensagem:

Mensagem 3 — WhatsApp

"Oi, filho! Meu celular quebrou e estou usando esse número novo. Preciso pagar uma conta urgente, consegue me fazer um PIX de R$ 150? Depois te devolvo. Não consigo atender ligação agora."

Exemplo de mensagem:

Mensagem 4 — Jogo online

"PARABÉNS! Você foi selecionado para receber 5.000 moedas grátis no seu jogo favorito! Acesse nosso site, entre com sua conta e resgate o prêmio antes que expire."

O que nessa mensagem faria vocês desconfiarem?

Phishing

Tentando enganar a vítima

Exemplo de mensagem:

Mensagem 5 — Professor

"Pessoal, a atividade de amanhã foi adiada para sexta-feira. Atualizei a data no calendário da turma. Qualquer dúvida, conversem comigo na aula."

Publicada pelo professor no ambiente virtual oficial da escola.

Exemplo de mensagem:

Mensagem 6 — Entrega

"Sua encomenda foi retida por uma taxa de entrega pendente de R$ 7,90. Pague imediatamente pelo link abaixo para evitar a devolução do produto."

Recebida por SMS de um número desconhecido.

O que nessa mensagem faria vocês desconfiarem?

Phishing

Tentando enganar a vítima

Exemplo de mensagem:

Mensagem 7 — Banco

"Uma tentativa de acesso à sua conta foi identificada. Para sua segurança, abra o aplicativo oficial do banco e consulte a seção de segurança. Nunca compartilhe sua senha ou código de verificação."

Notificação visualizada dentro do aplicativo bancário oficial.

Exemplo de mensagem:

Mensagem 8 — Bolsa de estudos

"Você foi selecionado para receber uma bolsa de estudos de R$ 1.200! Para confirmar sua vaga, envie uma foto do seu RG, CPF e comprovante de residência para este WhatsApp até as 18h."

O que nessa mensagem faria vocês desconfiarem?

ENGENHARIA SOCIAL

Phishing

Tentando enganar a vítima

Exemplo de mensagem:

Mensagem 9 — Loja online

"Seu pedido foi enviado! Você pode acompanhar a entrega diretamente na seção 'Meus Pedidos' do nosso aplicativo."

Exemplo de mensagem:

Mensagem 10 — Suporte técnico

"Olá! Somos da equipe de segurança da sua conta. Detectamos uma tentativa de invasão. Enviaremos um código por SMS. Por favor, responda esta mensagem com o código para confirmarmos sua identidade."

O que nessa mensagem faria vocês desconfiarem?

ENGENHARIA SOCIAL

Phishing

Tentando enganar a vítima

Exemplo de mensagem:

Mensagem 11 — Autenticação

"Seu código de verificação é 482913. Não compartilhe este código com ninguém. Se você não solicitou, ignore esta mensagem."

Exemplo de mensagem:

Mensagem 12 — Material de aula

"Boa tarde, turma! O material de revisão para a prova já está disponível na pasta 'Materiais' da plataforma oficial da escola. Bons estudos!"

O que nessa mensagem faria vocês desconfiarem?

Phishing ou Confiável?

Analise antes de clicar

Gabarito

1. Instagram — Suspeita
2. Escola — Parece confiável
3. WhatsApp — Suspeita
4. Jogo — Suspeita
5. Professor — Parece confiável
6. Entrega — Suspeita

Gabarito

7. Banco — Parece confiável
8. Bolsa — Suspeita
9. Loja — Parece confiável
10. Suporte — Suspeita
11. Autenticação — Parece confiável
12. Material — Parece confiável